

AWS Database Blog

Migrate from AIX or Windows Db2 to Amazon RDS for Db2

by Jitendra Kumar and Ashish Srivastava | on 20 JUL 2026 | in [Advanced \(300\)](#), [RDS for Db2](#), [Technical How-to](#) | [Permalink](#) | [Share](#)

Amazon Relational Database Service (Amazon RDS) for Db2 makes it straightforward to set up, operate, and scale IBM Db2 deployments in the cloud, automating time-consuming administration tasks such as provisioning, backups, software patching, and monitoring. Many organizations run IBM Db2 on AIX or Windows to support large-scale transactional and analytical workloads. These organizations are looking to reduce operational overhead by moving to this fully managed service, so they can focus on driving business value rather than managing infrastructure.

In this post, we provide step-by-step instructions for migrating an IBM Db2 database from on-premises AIX or Windows servers to Amazon RDS for Db2 using native Db2 tools (db2look and db2move). We also cover AWS Database Migration Service (AWS DMS) as an alternative approach. It also covers best practices to verify data integrity and minimize cutover time. With these practices, you can confidently transition to a managed service while maintaining full compatibility with your existing Db2 applications and workloads.

Reasons to migrate

Running IBM Db2 databases on a self-managed system like IBM AIX or Windows means you spend time and effort on hardware lifecycle management, operating system patching, backup and recovery operations, and manual infrastructure provisioning. These tasks divert your focus from higher-value activities like application optimization and innovation.

These on-premises environments present several ongoing issues:

- Maintenance overhead** – You must manage hardware refreshes, apply OS patches, and perform routine database maintenance. All of these tasks require dedicated effort and specialized expertise, increasing your operational costs.
- Scalability limitations** – Scaling compute and storage to meet fluctuating workload demands requires manual adjustments, often resulting in either over-provisioned infrastructure during low-demand periods or performance bottlenecks during peak loads.
- Cost inefficiencies** – Traditional on-premises deployments require initial capital investment and capacity planning for peak usage, leading to underutilized resources and inflated total cost of ownership.
- Complexity in disaster recovery (DR)** – Building and maintaining a robust disaster recovery strategy on-premises involves manually managing backup schedules, replication, and multisite failover configurations.
- Limited automation** – Routine operational tasks such as patching, backups, and monitoring lack built-in automation, requiring custom scripts or third-party tools that add complexity and maintenance burden.

Amazon RDS for Db2 solves these challenges by automating provisioning, patching, backups, and scaling. This automation helps database administrators (DBAs) manage up to [66% more databases](#) while reducing costs by up to [20% with Database Savings Plans](#). By migrating your Db2 workloads to this fully managed service, you alleviate the operational overhead of self-managed environments.

Solution overview

This post covers two migration methods. Choose the one that best fits your downtime window and operational requirements.

Method #1: Migrate data using AWS DMS

- Deploy Amazon RDS for Db2 in a private subnet within an Amazon [Virtual Private Cloud \(Amazon VPC\)](#).
- Run an AWS DMS replication instance within the same VPC to handle data migration from your on-premises source.
- Store database schema definitions and migration data in an [Amazon Simple Storage Service \(Amazon S3\)](#) bucket.
- Secure database credentials using [AWS Secrets Manager](#).
- Monitor AWS DMS task logs with [Amazon CloudWatch](#) to track migration progress.

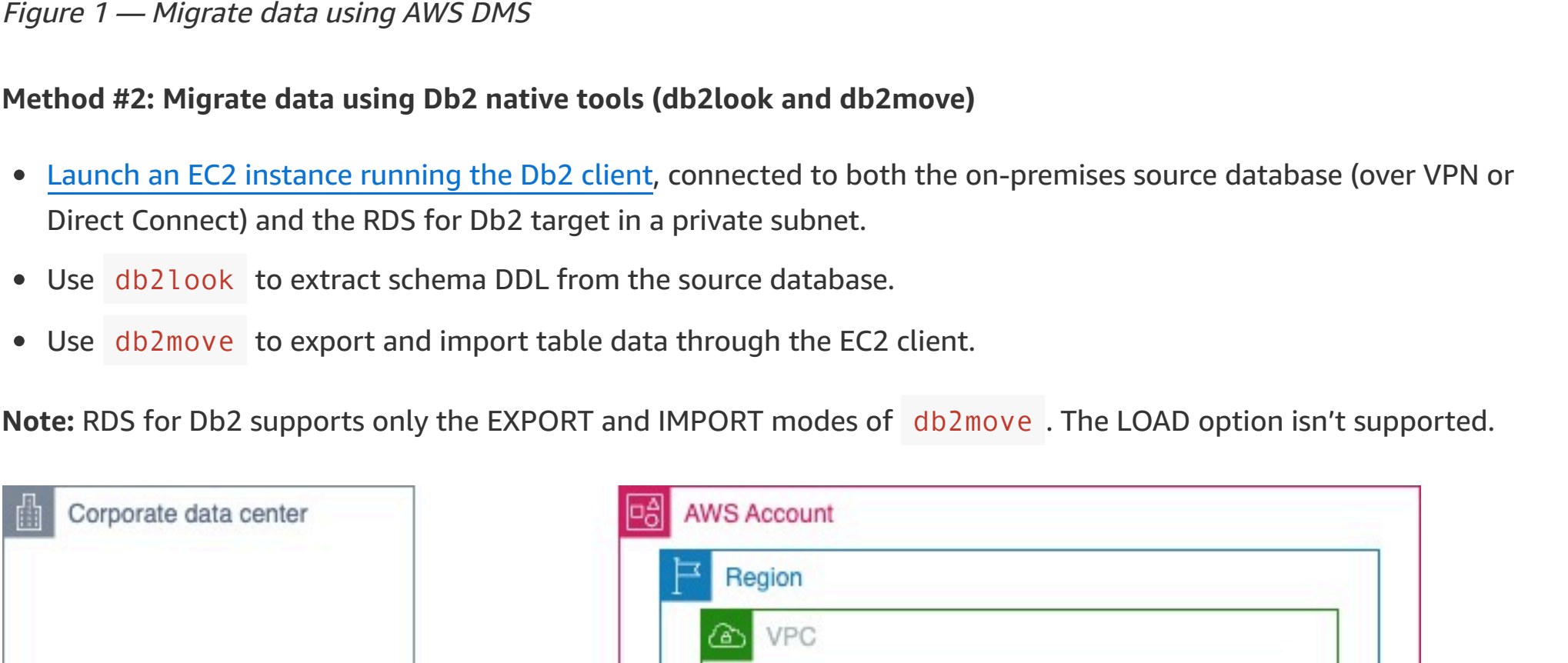


Figure 1 — Migrate data using AWS DMS

Method #2: Migrate data using Db2 native tools (db2look and db2move)

- [Launch an EC2 instance running the Db2 client](#), connected to both the on-premises source database (over VPN or Direct Connect) and the RDS for Db2 target in a private subnet.
- Use [db2look](#) to extract schema DDL from the source database.
- Use [db2move](#) to export and import table data through the EC2 client.

Note: RDS for Db2 supports only the EXPORT and IMPORT modes of [db2move](#). The LOAD option isn't supported.

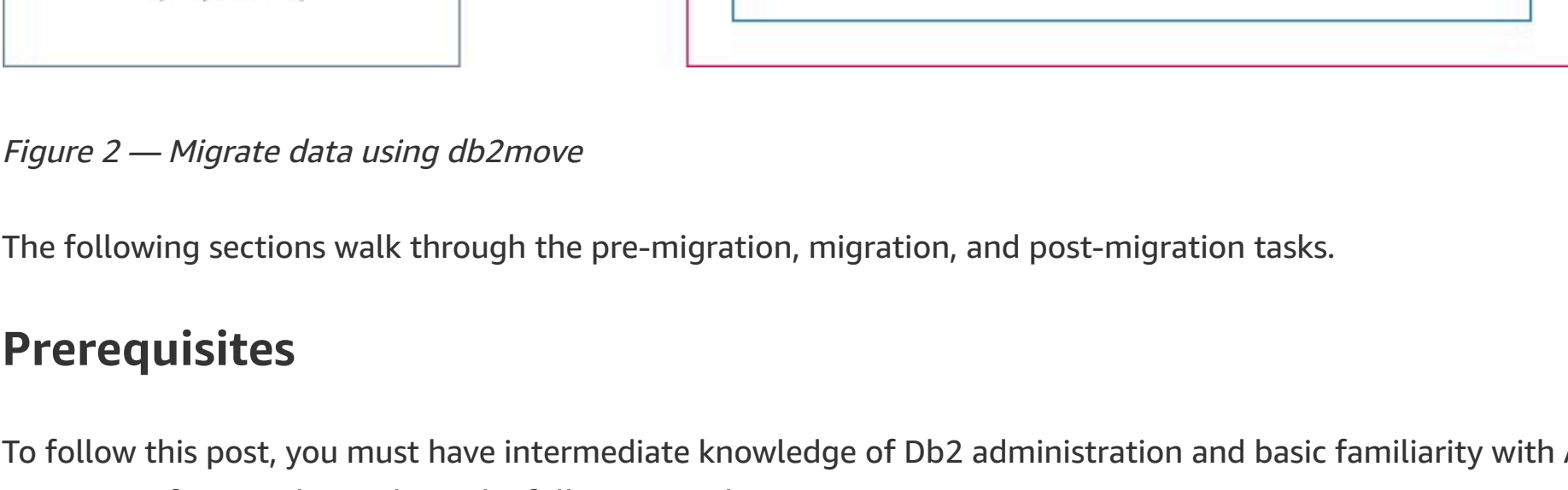


Figure 2 — Migrate data using db2move

The following sections walk through the pre-migration, migration, and post-migration tasks.

Prerequisites

To follow this post, you must have intermediate knowledge of Db2 administration and basic familiarity with AWS services. Before you begin, have the following in place:

1. Source database access and baseline.

Obtain database administrative authority (DBADM), the Db2 permission level required for schema and data export, to the source Db2 database on AIX/Windows. Establish current performance metrics as a baseline. Take a full backup of the source database (we recommend this for production systems). This backup serves as a critical recovery point. If problems occur during migration (such as data corruption, application compatibility problems, or unexpected failures), you can quickly restore the source database to its exact pre-migration state.

2. AWS infrastructure.

- AWS account:** Active account with appropriate [AWS Identity and Access Management \(AWS IAM\)](#) permissions ([AmazonRDSFullAccess](#) or a custom policy with required Amazon RDS permissions).
- Target database:** [Provision an Amazon RDS for Db2 instance](#).
 - Specify **db2** as the **engine type** when creating the DB instance.
- Network connectivity:** Secure connection between source environment and AWS (AWS Site-to-Site VPN or AWS Direct Connect).
- Migration tools.**

Set up and configure an AWS DMS replication instance for data replication. You can create the replication instance using the AWS Command Line Interface (AWS CLI) or AWS CloudFormation. For details, see [Creating a replication instance in the AWS DMS documentation](#).

If you plan to use Db2 native tools (db2move, db2look), follow these steps:

- [Set up Db2 client](#) on [AWS CloudShell](#) or an [Amazon Elastic Compute Cloud \(Amazon EC2\)](#) instance. This instance should be in the same Amazon VPC as your RDS for Db2 DB instance.
- Check that the EC2 instance can connect to both the source and target databases.
- [Catalog](#) source and target databases on EC2.
- Application inventory and access configuration.**

Create a complete inventory of applications connecting to the database, including their connection requirements and authentication methods. Configure the target RDS for Db2 instance to match source system access patterns (for example, [enable Kerberos authentication](#) if used in the source environment).

5. Check the Amazon RDS for Db2 limitations.

Check the [Known issue and limitations](#) for Amazon RDS for Db2 before proceeding.

Pre-migration activities

You can perform the following pre-migration activities 1–2 weeks before the actual migration to reduce the production downtime window.

Perform these activities on the source database:

- Connect to the source server and run the [db2look](#) command to extract the Data Definition Language (DDL) statements.

```
export OUTPUT_DIR=/path/to/your/directory/<ddl>
export DATABASE_NAME=<source_db>
export OUTPUT_FILE=${OUTPUT_DIR}/${DATABASE_NAME}.sql
echo "date" "Exporting the DDL of ${DATABASE_NAME} database. The output will be saved in"
db2look -d ${DATABASE_NAME} -e -a -x -o ${OUTPUT_FILE}
db2look -d ${DATABASE_NAME} -wlm db2look_wlm.sql
```

Where:

- d <database_name>** : Is the alias name of the database that's to be migrated.
 - a** : Generates DDL statements for objects that were created by each database user, including inoperative objects.
 - e** : Extracts DDL statements.
 - x** : Generates authorization DDL statements such as GRANT statements.
 - o** : Writes the output to a **<source_db>.sql** file.
 - wlm** : Generates WLM (workload manager) specific DDL output.
- The [db2look](#) command generates DDL statements by object type.
- Go through the output file, **<source_db>.sql**, and separate the foreign keys and the triggers definitions.
 - Save them to separate files, as **<source_db>_foreignkey.sql** and **<source_db>_trigger.sql** respectively.

Perform these activities on the target database:

- Catalog RDS for Db2 node and connect to the **rdsadmin** database using master user credentials. The **rdsadmin** database is the RDS for Db2 system database used for administrative operations.
- The **<rds_username>** master user was specified during instance creation.

```
sudo su - <instance_owner>
db2 catalog tcpip node <node> remote <rds-db2-endpoint> server <port>
db2 catalog database rdsadmin at node <node>
db2 connect to rdsadmin user <rds_username>
```

- RDS for Db2 offers a set of stored procedures for database management. For a complete list, see the [Amazon RDS for Db2 stored procedure reference](#).
- To create the target database, use the **rdsadmin.create_database** stored procedure. This is the standard mechanism for creating databases within your RDS for Db2 DB instance.

```
db2 "call rdsadmin.create_database(
'database_name',
'database_page_size',
'database_code_set',
'database_territory',
'database_collation',
'database_autoconfigure_str',
'database_non-restrictive')"
```

Make sure that the database parameters, such as **database_page_size**, **database_code_set**, **database_territory**, and **database_collation**, match the source database configuration to maintain compatibility during migration.

5. Catalog the target database.

```
db2 catalog database <target_db> as <target_db_alias> at node rdsdb2 authentication server
```

6. Use [built-in stored procedures](#) to create the buffer pools, tablespaces, database users/roles in the target database.

7. Open the **<source_db>.sql** script and remove the create command for buffer pools, tablespaces, database users/roles.

8. Run the following commands to create database objects such as tables and indexes.

```
# Connect to Amazon RDS for Db2 instance
db2 connect to <target_db> user <rds_username>

# Apply the schema
db2 -tvf <source_db>.sql
```

Note: To avoid errors, don't run **<source_db>_foreignkey.sql** and **<source_db>_trigger.sql** before importing the data.

9. Validate that the database objects have been created.

Migration activities

You can choose from two methods to migrate the data. Your choice depends on your available downtime window, database size, and whether you need the source database to remain operational during migration:

Method #1: Migrate data using AWS DMS

We recommend AWS DMS when your downtime window is limited relative to the data volume, or when you need change data capture (CDC) to keep the source and target in sync during migration. AWS DMS performs a full-load migration and then replicates ongoing changes from the source database, ending when you complete the cutover to Amazon RDS for Db2. For detailed steps, see the AWS blog post [Migrate from self-managed Db2 to Amazon RDS for Db2 using AWS DMS](#).

Method #2: Migrate data using db2move

The [db2move](#) utility is a good choice when you can schedule a maintenance window long enough for a full export/import cycle, or when you need fine-grained control over which tables and schemas to migrate. This approach doesn't require provisioning a DMS replication instance, reducing infrastructure complexity.

Note that [db2move](#) exports table data only. It doesn't export views, stored procedures, functions, triggers, or sequences. That's why the [db2look](#) DDL extraction in the pre-migration step is required to recreate these objects on the target.

Export data from source

```
# Create export directory
mkdir /path/to/your/directory/<data>

# Run db2move command to export data
db2move <source_db> export -sn <schema_name>
```

Note: Consider using the following additional flags with [db2move](#) :

- l <lobpath>** : Specifies a separate directory for exporting/importing LOB (Large Object) data. If your tables contain BLOB, CLOB, or DBCLOB columns, use this flag to direct LOB files to a path with sufficient disk space.
- aw** (allow warnings): Allows the import operation to continue when non-critical warnings are encountered (for example, row-level truncation warnings for trailing blanks). Without this flag, [db2move](#) import stops on any warning, which can interrupt large migrations unnecessarily.

```
db2move <source_db> EXPORT -sn <schema_name>
```

For the full list of options, see the [IBM db2move documentation](#).

Import data to Amazon RDS for Db2

Imports all tables listed in the **db2move.lst** internal staging file. Use the **-io** option for IMPORT-specific actions.

- io <import_option>** : Specifies the import mode.

Valid options are **INSERT**, **INSERT_UPDATE**, **REPLACE**, **CREATE**, and **REPLACE_CREATE**. The default is **REPLACE_CREATE**.

Note: For migrations to RDS for Db2, we recommend using **INSERT** or **REPLACE** because the target tables are already created using the [db2look](#) DDL script in the pre-migration step.

```
# Go to the same directory where you stored the exported file
cd /path/to/your/directory/<data>

# Run db2move command to import data into target
db2move <target_db> IMPORT -u <rds_username> -p <rds_username_password>
```

Post-migration activities

After completing the data migration, run the following tasks in order:

- Collect statistics and optimize table layout. Run these steps in order:

- Run the [RUNSTATS](#) utility on the tables and indexes. This provides the Db2 query optimizer with up-to-date statistics, enabling it to generate effective query execution plans.

```
RUNSTATS ON TABLE <schema>.<table> WITH DISTRIBUTION AND DETAILED INDEXES ALL;
```

- Run [REORGCHK](#) to identify tables or indexes that might benefit from reorganization:

```
db2 reorgchk current statistics on table all
```

- If reorgchk recommends reorganization, run REORG on those tables:

```
REORG TABLE <schema>.<table>;
REORG INDEXES ALL FOR TABLE <schema>.<table>;
```

- Run RUNSTATS again only on tables you reorganized.

- Run the **<dbname>_foreignkey.sql** and **<dbname>_trigger.sql** scripts separately to reestablish the foreign key relationships and triggers.

```
db2 -tvf <source_db>_foreignkey.sql
db2 -tvf <source_db>_trigger.sql
```

- Next, verify data integrity and completeness by comparing row counts, checksums, and data consistency between your source and target databases to confirm that the migration transferred your data correctly.
- Confirm that the schemas, tables, indexes, views, stored procedures, functions, and constraints between migrated correctly and match the source database. For a detailed validation approach, see the AWS blog post [Validate database object consistency after migrating from IBM Db2 to Amazon RDS for Db2](#).
- For security, validate that the users, roles, and permissions are correctly configured and that your applications can connect with the appropriate access levels.
- Update your application configuration to point to the new Amazon RDS for Db2 endpoint and test connectivity from each application. For more information, see the [validate connectivity](#) guide.
- [Set up Amazon CloudWatch](#) metrics, alarms, and Enhanced Monitoring so you can proactively track database health going forward.

Limitations

For additional constraints and known issues, see [Known issues and limitations for Amazon RDS for Db2](#) in the AWS documentation.

Best practices

Follow these best practices to help your migration succeed:

- We recommend running a test migration before migrating production data. Use a non-production environment to identify potential issues, validate the migration process, and establish realistic time estimates for your cutover window.
- We recommend monitoring Amazon RDS performance using Amazon CloudWatch metrics, including CPU utilization, storage, IOPS (I/O operations per second), and connection counts. Set up alarms for key thresholds.
- For security, we recommend configuring security groups that restrict database access to authorized users only, enabling encryption at rest and in transit, storing credentials in AWS Secrets Manager, and using AWS Identity and Access Management (AWS IAM) roles for access control.
- For high availability (HA), we recommend deploying Amazon RDS instances across multiple Availability Zones (AZs). Verify that AWS DMS replication instances have sufficient subnet and security group access to both source and target databases.
- We recommend configuring appropriate AWS DMS task settings for full-load migration. Monitor replication lag and use table mappings to improve migration performance.
- For capacity planning, we recommend selecting the appropriate Amazon RDS instance size (based on your workload requirements. Factor in the cost of running an AWS DMS replication instance during migration (terminate after cutover), Amazon RDS instance pricing (on-demand compared to reserved), and Amazon S3 storage costs for migration data. Plan for storage growth using Amazon CloudWatch metrics and historical data.

Clean up resources

To avoid incurring ongoing charges for resources that are no longer needed, delete the resources created during this migration by completing the following steps:

Delete AWS DMS resources

- Use the AWS CLI or the AWS DMS console to delete the migration task.
- Delete the source and target endpoints.
- Delete the replication instance.
- Choose **Subnet groups** and delete the AWS DMS subnet group if you created one specifically for this migration.

Delete Amazon RDS for Db2 database

- Use the AWS CLI or the Amazon RDS console to delete the DB instance.
- Select the Amazon RDS for Db2 database that you created for this migration.
- Confirm deletion and acknowledge that automated backups will no longer occur.
- Enter **delete me** to confirm deletion.
- Choose **Delete**.

Delete supporting resources

- Delete the Amazon S3 bucket and its contents from the Amazon S3 console.
- Delete the database credentials from AWS Secrets Manager.
- Delete Amazon CloudWatch log groups associated with AWS DMS tasks.
- Remove security groups created for this migration (if no longer needed).

For more information, see:

- [Deleting a DB instance](#)
- [Working with AWS DMS tasks](#)

Conclusion

In this post, you migrated an IBM Db2 database from AIX or Windows to Amazon RDS for Db2 using native Db2 tools ([db2look](#) and [db2move](#)) and AWS DMS. After completing the migration, you can dedicate more time to database improvements, capacity planning, and performance tuning while Amazon RDS for Db2 handles patching, backups, and failover automatically.

To begin your migration: (1) assess your current database size to determine whether AWS DMS or [db2move](#) is the right fit, (2) provision an Amazon RDS for Db2 instance in a non-production environment, and (3) run a test migration following the steps in this post. For more information, see the [Amazon RDS for Db2 documentation](#) and the [AWS DMS documentation](#).

About the authors

Jitendra Kumar
Jitendra is a Senior Delivery Consultant, specializing in enterprise databases such as IBM Db2, SAP ASE, PostgreSQL and SQL Server. With over 20 years of experience, he helps customers plan and execute database migrations and modernization on AWS.

Ashish Srivastava
Ashish is a Delivery Consultant – Data & Analytics with the Professional Services team at Amazon Web Services. Ashish works as a database migration specialist with experience in enterprise databases like Db2 and SQL Server, providing technical guidance to help you migrate your on-premises databases to AWS.

Resources

- [Amazon Aurora](#)
- [Amazon DocumentDB](#)
- [Amazon DynamoDB](#)
- [Amazon ElastiCache](#)
- [Amazon Keyspaces \(for Apache Cassandra\)](#)
- [Amazon MemoryDB](#)
- [Amazon Neptune](#)
- [Amazon Neptune Analytics](#)
- [Amazon RDS](#)
- [Amazon Timestream](#)
- [AWS Database Migration Service](#)